Provisional Patent Application — IDENTITYOS-PROV-001

PROVISIONAL PATENT APPLICATION

Docket No.: IDENTITYOS-PROV-001 **Filing Type:** Provisional Application for Patent (35 USC 111(b)) **Entity Status:** Small Entity **Inventor:** Viswanadha Pratap Kondoju, Charlotte, North Carolina, USA

TITLE OF THE INVENTION

Privacy-Preserving Mutual Authentication and Composable Delegation Between Human and Artificial Intelligence Agent Identities Using Zero-Knowledge Proofs with Scope Commitment Chain Linking

---

CROSS-REFERENCE TO RELATED APPLICATIONS

This application is related to but distinct from the inventor's existing patent portfolio comprising twenty-three (23) applications directed to ZKP-powered portable credit union identity (ZKProva, PROV-001 through PROV-009 and associated CIPs) and three (3) applications directed to GENIUS Act compliance infrastructure (GeniusComply, PROV-010, CIP-010-A, PROV-011). The present invention addresses a different technical problem — unified identity authentication and delegation across human and artificial intelligence agent populations — using different circuit architectures, a mixed proving system design, and novel scope commitment chain primitives not disclosed in the related applications.

ABSTRACT

A computer-implemented system and method for privacy-preserving mutual authentication between a human user and an artificial intelligence (AI) agent, and for composable delegation of scoped permissions through chains of agents, using zero-knowledge proofs (ZKPs). The system maintains separate Merkle trees for human identity commitments and AI agent credential commitments on a blockchain-based registry. A mutual handshake protocol enables a human and an AI agent to independently generate zero-knowledge proofs — a Groth16 proof for human group membership and a PLONK proof for agent credential validity — that are verified in a single on-chain transaction with explicit session nonce equality enforcement across both proofs, without either party learning anything beyond policy satisfaction. A delegation circuit enables privacy-preserving permission narrowing through multi-hop agent chains by

chaining identity-bound Poseidon hash commitments that bind both permission bitmasks and credential commitments together, where each delegation hop proves that the delegatee's scope is a bitwise subset of the delegator's scope without revealing the actual permission bits to any on-chain observer, and where delegation nullifiers, chain-linking verification, and maximum hop count enforcement prevent replay and impersonation attacks. A cumulative bit encoding scheme for hierarchical permissions ensures that higher-tier financial permissions cryptographically imply lower-tier permissions, with circuit constraints enforcing the invariant at delegation time.

BACKGROUND OF THE INVENTION

Field of the Invention

The present invention relates generally to cryptographic identity protocols, and more specifically to systems and methods for mutual zero-knowledge proof authentication between human users and artificial intelligence agents, with privacy-preserving composable delegation of scoped permissions.

Description of Related Art

The proliferation of artificial intelligence agents operating autonomously on behalf of human users has created an urgent need for identity and authorization infrastructure that spans both human and AI populations. Current approaches suffer from fundamental limitations:

Hardware-Dependent Human Identity Systems. World (formerly Worldcoin) provides proof-of-personhood through iris scanning using specialized Orb hardware. While cryptographically sound, this approach requires physical hardware distribution, creates single points of failure, and couples identity establishment to biometric data capture. World's AgentKit extension binds AI agents to World IDs but does not support privacy-preserving sub-delegation — an agent authorized by a human cannot further delegate narrower permissions to a subordinate agent without exposing the delegation structure.

Government-ID-Dependent Systems. Systems such as Didit rely on government-issued identification documents for identity verification. These systems are inherently limited by the availability and reliability of government ID infrastructure, exclude populations without such documentation, and expose personal identifying information during verification.

Social Graph Systems. BrightID employs social graph analysis to establish uniqueness. This approach exposes social connection patterns, is vulnerable to Sybil attacks through coordinated social engineering, and does not extend to AI agent identity.

Verifiable Credential Systems Without Privacy. Indicio's ProvenAI issues W3C Verifiable Credentials for AI agents, providing operator attestation and capability claims. However, these credentials are presented in cleartext — the verifier learns the agent's model, operator, and full permission set. There is no zero-knowledge layer to prove credential properties without revealing the credential itself.

Anonymous Group Membership Without Agent Support. The Semaphore Protocol (by the Privacy and Scaling Explorations group at the Ethereum Foundation) provides anonymous group membership proofs using zero-knowledge proofs over Merkle trees. Semaphore enables a member to prove they belong to a group without revealing which member they are, using EdDSA identity commitments on the Baby Jubjub elliptic curve, Poseidon hash functions, and Groth16 SNARKs. However, Semaphore addresses only single-population anonymity — it has no concept of AI agent identity, mutual authentication between heterogeneous entity types, or delegation of permissions.

Blockchain Analytics Without Privacy. On-chain identity systems that rely on wallet history, token holdings, or transaction patterns expose all identity information publicly. These approaches are fundamentally incompatible with privacy-preserving identity.

Capability Delegation Systems Without Privacy. UCAN (User Controlled Authorization Networks) and Biscuit provide capability attenuation chains where each delegation narrows permissions. However, these systems operate on cleartext tokens — every party in the chain and every verifier can read the full permission structure, delegation history, and party identities. There is no zero-knowledge layer; privacy is achieved only by not sharing the token, which fails when on-chain verification is required.

Privacy-Preserving Credentials Without Agent Support. AnonCreds (Hyperledger) and BBS+ selective disclosure (W3C) enable holders to prove credential properties without revealing the full credential. However, these systems address single-holder credential presentation, not mutual authentication between heterogeneous entity types, and do not support composable delegation chains where permissions narrow at each hop.

Rate-Limiting Nullifiers. The RLN (Rate-Limiting Nullifier) protocol uses per-epoch nullifiers to prevent spam in anonymous systems. While RLN shares the nullifier primitive with the present invention, it addresses message rate-limiting in a single homogeneous population, not mutual authentication or permission delegation across human and AI agent populations.

None of the existing approaches provide: (1) a unified protocol for both human and AI agent identity; (2) mutual authentication where both parties simultaneously prove claims without learning each other's identity; (3) privacy-preserving delegation chains where permissions narrow at each hop without revealing the actual permission structure, with scope commitments bound to both permissions and delegator identity; or (4) a mixed proving system architecture that optimizes for different trust assumptions on the human and agent sides.

SUMMARY OF THE INVENTION

The present invention provides a system and method for privacy-preserving mutual authentication and composable delegation between human and AI agent identities. The system comprises:

First, a mutual zero-knowledge proof handshake protocol in which a human user and an AI agent independently generate cryptographic proofs — the human proving group membership via a Groth16 SNARK and the agent proving credential validity via a PLONK SNARK — that

are verified in a single blockchain transaction with explicit enforcement that both proofs embed the same session nonce (the on-chain verifier checks equality between each proof's public nonce signal and the transaction argument). Neither party learns anything about the other beyond the fact that the counterparty satisfies the required policy.

Second, a privacy-preserving composable delegation chain mechanism in which scoped permissions are delegated from an agent to subsequent agents, using zero-knowledge proofs. At each delegation hop, an identity-bound Poseidon hash commitment of the delegatee's permission bitmask and credential commitment is published as a public output, while the actual permission bits remain private. The next hop's circuit takes the previous commitment as a public input and proves that (a) the Poseidon hash of the delegator's actual scope together with the delegator's credential commitment equals the previous commitment (binding scope to identity), (b) the delegatee's scope is a bitwise subset of the delegator's scope, and (c) the delegatee's expiry does not exceed the delegator's expiry — all without revealing any party's actual permission bits. On-chain verification enforces delegation nullifier uniqueness, chain-linking correctness, and a maximum hop count of three.

Third, a cumulative bit encoding scheme for hierarchical permissions in which higher-tier permission bits cryptographically imply the presence of lower-tier bits (e.g., bit 4 for unlimited financial transactions implies bits 2 and 3 for lower-tier financial transactions). Circuit constraints enforce this invariant at delegation time, ensuring that delegation narrowing via bitwise AND correctly expresses tier downgrades without creating inconsistent permission states.

Fourth, a mixed proving system architecture that uses Groth16 (with trusted setup ceremony reuse from Semaphore v4) for the human identity circuit and PLONK (with universal setup requiring no circuit-specific ceremony) for the agent credential and delegation circuits. An on-chain registry contract manages separate Lean Incremental Merkle Trees for human and agent populations and verifies both proof types within a single transaction through distinct verifier contract interfaces.

DETAILED DESCRIPTION OF THE INVENTION

1. System Architecture Overview

Referring now to Figure 1, the IdentityOS system 100 comprises three principal components: (a) a human identity subsystem 110 that manages human identity commitments in a humanTree Merkle structure; (b) an AI agent credential subsystem 120 that manages agent credential commitments in an agentTree Merkle structure; and (c) an on-chain IdentityRegistry contract 130 deployed on a Layer 2 blockchain (e.g., Base) that stores both Merkle trees, verifies proofs, tracks nonce freshness, and manages revocation state.

```
FIGURE 1: System Architecture

+-------------------------------------------------------------+
|                     IdentityOS System (100)                  |
|                                                              |
```

```
|  +-------------------+          +----------------------+    |
|  | Human Identity    |          | Agent Credential     |    |
|  | Subsystem (110)   |          | Subsystem (120)      |    |
|  |                   |          |                      |    |
|  | - EdDSA keypair   |          | - Credential fields  |    |
|  |   (Baby Jubjub)   |          |   (model, operator,  |    |
|  | - Identity commit |          |    permissions,      |    |
|  |   = Poseidon2(    |          |    expiry)           |    |
|  |     Ax, Ay)       |          | - Credential commit  |    |
|  | - Groth16 prover  |          |   = Poseidon5(...)   |    |
|  |                   |          | - PLONK prover       |    |
|  +--------+----------+          +----------+-----------+    |
|           |                                |                |
|           |   humanProof (Groth16)         | agentProof     |
|           |                                | (PLONK)        |
|           v                                v                |
|  +-------------------------------------------------------+  |
|  |           IdentityRegistry Contract (130)             |  |
|  |                                                       |  |
|  |  humanTree (LeanIMT, depth 20)                        |  |
|  |  agentTree (LeanIMT, depth 20)                        |  |
|  |  rootHistory[30] (agent root buffer)                  |  |
|  |  usedNonces mapping (replay protection)               |  |
|  |  revocation mappings (human + agent)                  |  |
|  |                                                       |  |
|  |  verifyHandshake(humanProof, agentProof, nonce)       |  |
|  |  verifyDelegation(proof, pubSignals, nonce)           |  |
|  +-------------------------------------------------------+  |
|                          |                                  |
|                     Layer 2 Blockchain                      |
+-------------------------------------------------------------+
```

2. Human Identity Circuit (HumanUniqueness)

The HumanUniqueness circuit 200 is a Groth16 zero-knowledge proof circuit compiled using Circom 2.1.6 with a Merkle tree depth of 20 (supporting approximately one million human identities). The circuit is designed to be compatible with the Semaphore v4 identity scheme, enabling reuse of the Semaphore v4 Powers of Tau Phase 1 trusted setup ceremony.

FIGURE 2: HumanUniqueness Circuit (200)

```
PRIVATE INPUTS                PUBLIC INPUTS          PUBLIC OUTPUTS
+----------------+           +---------------+       +-------------------+
| secret         |           | scope         |       | humanMerkleRoot   |
| (EdDSA scalar) +--+        | sessionNonce  |       | nullifierHash     |
+----------------+  |        +-------+-------+       | nonceBinding      |
                    |                |               +-------------------+
                    v                |                       ^
              +-----------+          |                       |
              | BabyPbk() |          |                       |
              | secret -> |          |                       |
              | (Ax, Ay)  |          |                       |
              +-----+-----+          |                       |
                    |                |                       |
                    v                |                       |
           +----------------+      |                      |
```

```
          | Poseidon2(     |      |                        |
          |   Ax, Ay)      |      |                        |
          | = identityCmt  |      |                        |
          +-------+--------+      |                        |
                  |               |                        |
        +--------+--------+     |                       |
        |                 |     |                       |
        v                 v     v                       |
+---------------+  +--------------+                    |
| BinaryMerkle  |  | Poseidon2(   |                    |
| Root(depth=20)|  |  scope,      |                    |
| leaf=idCmt    |  |  secret)     |                    |
| -> root       |  |  = nullifier |                    |
+-------+-------+  +------+-------+                    |
        |                 |                            |
        |                 +-----> Poseidon2(           |
        |                        nullifier,            |
        |                        sessionNonce)         |
        |                        = nonceBinding --------+
        |                                              |
        +----------------------------------------------+
```

Step 2.1 — Key Derivation: The circuit receives the human's EdDSA secret scalar as a private input. Using the BabyPbk (Baby Jubjub point multiplication) component from circomlib, the circuit derives the public key point (Ax, Ay) on the Baby Jubjub elliptic curve. A Num2Bits(251) range check provides a conservative upper bound on the secret, constraining it to be less than 2^251. This bound is strictly less than the Baby Jubjub subgroup order (l = 2736030358979909402780800718157159386076813972158567259200215660948447373041), so all valid secrets pass. The conservative bound avoids the ~500 extra constraints required for an exact multi-limb comparison against l.

Step 2.2 — Identity Commitment: The identity commitment is computed as `Poseidon2(Ax, Ay)`, producing a field element that serves as the leaf value in the humanTree Merkle structure. This computation is compatible with the `@semaphore-protocol/identity` library's commitment scheme.

Step 2.3 — Merkle Membership Proof: Using the `BinaryMerkleRoot` component from `@zk-kit/binary-merkle-root.circom`, the circuit proves that the identity commitment is a leaf in the humanTree. The circuit takes a Merkle proof (sibling hashes, proof index, and proof length) as private inputs and outputs the computed Merkle root as a public output. The on-chain verifier compares this root against the current humanTree root stored in the IdentityRegistry contract.

Step 2.4 — Nullifier Computation: The nullifier is computed as `Poseidon2(scope, secret)`, where `scope` is a public input identifying the context (e.g., `hash("identityos-handshake-v1")`). This produces a deterministic value per identity per scope, enabling Sybil detection: the same identity produces the same nullifier within the same scope, regardless of how many proofs it generates.

Step 2.5 — Nonce Binding: The nonce binding is computed as `Poseidon2(nullifierHash, sessionNonce)`, where `sessionNonce` is a verifier-generated public input. This binds the proof to a specific session, preventing replay attacks. The on-chain verifier checks that the session nonce has not been previously used.

The compiled circuit has 16,409 R1CS constraints and generates proofs in approximately 0.57 seconds on an Apple M-series processor using Groth16.

3. AI Agent Credential Circuit (AgentPolicy)

The AgentPolicy circuit 300 is a PLONK zero-knowledge proof circuit compiled using Circom 2.1.6 with a Merkle tree depth of 20 (supporting approximately one million agent credentials).

FIGURE 3: AgentPolicy Circuit (300)

```
PRIVATE INPUTS                     PUBLIC INPUTS         PUBLIC OUTPUTS
+---------------------+          +----------------+     +------------------+
| modelHash           |          | requiredScope  |     | agentMerkleRoot  |
| operatorPubkeyAx    |          |   Mask         |     | nullifierHash    |
| operatorPubkeyAy    |          | currentTime    |     | scopeCommitment  |
| permissionBitmask   |          |   stamp        |     +------------------+
| expiryTimestamp      |          | sessionNonce   |
| sigR8x, sigR8y, sigS|          +-------+--------+
| merkleProof[20]      |                  |
+----------+-----------+                  |
           |                              |
   +-------v--------+                    |
   | Num2Bits(64)   |   (range check     |
   | bitmask, expiry|    all uint64s)    |
   | timestamp      |                    |
   +-------+--------+                    |
           |                              |
   +-------v-----------+                 |
   | Poseidon5(         |                |
   |   modelHash,       |                |
   |   operatorAx,      |                |
   |   operatorAy,      |                |
   |   bitmask,         |                |
   |   expiry)          |                |
   | = credentialCmt    |                |
   +---+-------+--------+               |
       |       |                         |
       v       v                         |
+----------+ +-------------------+       |
| EdDSA    | | BinaryMerkleRoot  |       |
| Poseidon | | (depth=20)        |       |
| Verify   | | leaf=credCmt      |       |
| (opSig   | | -> agentRoot  ----+------+---> agentMerkleRoot
|  over    | +-------------------+       |
|  credCmt)|                             |
+----------+                             |
       |            +-----------+        |
       |            | Bit-by-bit|        |
       |            | scope     |        |
       |            | check:    |        |
       |            | req[i] *  |        |
       |            | (1-perm[i])        |
       |            | == 0      |        |
       |            +-----------+        |
       |                                 |
       |     +------------------+        |
```

```
|     | LessThan(64)    |       |
|     | currentTs <     |       |
|     |  expiryTs       |       |
|     +-----------------+       |
|                               |
+-> Poseidon2(credCmt, nonce)   |
    = nullifierHash  -----------+
                                |
    Poseidon2(bitmask, credCmt) |
    = scopeCommitment  ---------+
```

Step 3.1 — Range Checks: Three Num2Bits(64) components decompose `permissionBitmask`, `expiryTimestamp`, and `currentTimestamp` into their binary representations, constraining each to be a valid 64-bit unsigned integer. This prevents field overflow attacks where a value exceeding 2^64 passes the circuit's arithmetic checks but overflows when processed by the Solidity verifier contract.

Step 3.2 — Credential Commitment: The credential commitment is computed as `Poseidon5(modelHash, operatorPubkeyAx, operatorPubkeyAy, permissionBitmask, expiryTimestamp)`. This five-field Poseidon hash produces a single field element that uniquely commits to the agent's credential properties, including both coordinates of the operator's EdDSA public key to fully bind the operator's identity. The credential commitment serves as the leaf in the agentTree Merkle structure.

Step 3.3 — Operator Signature Verification: Using the `EdDSAPoseidonVerifier` component from circomlib, the circuit verifies that the operator's EdDSA signature (R8x, R8y, S) over the credential commitment is valid for the operator's public key (operatorPubkeyAx, operatorPubkeyAy). This proves that a recognized operator authorized the agent's credential — the operator signed an attestation of the agent's model, permissions, and expiry.

Step 3.4 — Merkle Membership Proof: Identical in structure to the human circuit's Merkle proof (Step 2.3), but operating over the agentTree. The computed Merkle root is output as a public signal and verified against a root history buffer of the last 30 agentTree roots maintained in the IdentityRegistry contract. The root history buffer accommodates proof staleness — if the agentTree is updated (new agents enrolled) between the time an agent generates its proof and the time the proof is verified on-chain, the proof remains valid as long as the root it references is within the last 30 updates.

Step 3.5 — Permission Scope Check: The circuit performs a bit-by-bit scope verification. For each of the 64 bit positions, the circuit constrains: `requiredBits[i] * (1 - permissionBits[i]) === 0`. This enforces that every bit set in the `requiredScopeMask` (public input from the verifier's policy) must also be set in the agent's `permissionBitmask` (private input). The agent's full permission set remains private — the verifier learns only that the agent satisfies the required scope, not what additional permissions the agent holds.

Step 3.6 — Expiry Check: A `LessThan(64)` comparator constrains `currentTimestamp < expiryTimestamp`, proving the credential has not expired. The `currentTimestamp` is a public input provided by the verifier or relayer.

Step 3.7 — Nullifier and Scope Commitment: The nullifier is computed as `Poseidon2(credentialCommitment, sessionNonce)`, providing per-session replay protection. The scope commitment is computed as `Poseidon2(permissionBitmask, credentialCommitment)` and is output as a public signal. By binding the permission bitmask to the credential commitment, the scope commitment becomes identity-bound. As used herein, a commitment is said to be "identity-bound" when its cryptographic pre-image includes both a policy field (e.g., a permission bitmask representing the entity's authorized capabilities) and an identity field (e.g., a credential commitment that is Merkle-included in an on-chain identity tree), such that the commitment cannot be reproduced by any party not in possession of both fields. An actor with the same permission bits but a different credential cannot produce a matching identity-bound scope commitment, and therefore cannot satisfy the chain-linking check in the Delegation circuit (Section 5). This prevents impersonation attacks in delegation chains.

Step 3.8 — Scope Commitment Privacy Limitations and Mitigations: The scope commitment `Poseidon2(permissionBitmask, credentialCommitment)` is privacy-hiding within the ZK protocol: no party learns the actual bitmask from the proof. However, the `permissionBitmask` is a 64-bit value with a structured domain (many bits are typically zero), and the `credentialCommitment` is public (it is a leaf in the on-chain agentTree). An offline attacker who knows the credentialCommitment can enumerate all 2^64 possible bitmask values and precompute the corresponding Poseidon hashes in approximately 2^64 hash evaluations. On modern GPUs this is feasible for a determined adversary. Mitigations in alternative embodiments include: (a) **entropy injection** — concatenating a random blinding salt with the bitmask before hashing, producing `scopeCommitment = Poseidon3(permissionBitmask, credentialCommitment, blindingSalt)` where `blindingSalt` is a private circuit input; (b) **domain restriction** — limiting the public visibility of scope commitments to on-chain events rather than persistent storage, reducing the window for offline enumeration; (c) **scope grouping** — hashing permission groups rather than individual bitmasks to reduce the search space's structure. The preferred embodiment uses the unblinded `Poseidon2` construction because the credentialCommitment is already publicly known (Merkle-included), and the primary threat model (preventing impersonation in delegation chains) is addressed by the identity-binding property rather than by hiding the scope from offline enumeration.

The compiled circuit has 20,832 R1CS constraints and generates proofs in approximately 16.3 seconds using PLONK.

4. Mutual Handshake Protocol

The mutual handshake protocol 400 is the core authentication primitive. It enables a human and an AI agent to mutually authenticate without learning each other's identity or credential details.

```
FIGURE 4: Mutual Handshake Protocol (400)

   HUMAN (110)                                    AGENT (120)
       |                                              |
       |  1. Agree on sessionNonce (random, fresh)    |
       |<---------------------------------------------->|
       |                                              |
       |  2. Generate Groth16 proof                   |  3. Generate PLONK
```

```
proof
       |     (HumanUniqueness circuit)                |     (AgentPolicy
circuit)
       |     Private: secret, merkleProof             |     Private:
credential,
       |     Public: scope, nonce                     |       sig, merkleProof
       |     Output: root, nullifier,                 |     Public: scope,
time,
       |       nonceBinding                           |       nonce
       |                                              |     Output: root,
nullifier,
       |                                              |       scopeCommitment
       v                                              v
  +-------------------------------------------------------+
  |        IdentityRegistry.verifyHandshake()  (130)      |
  |                                                       |
  |  4. Check: nonce not in usedNonces                    |
  |  5. Mark nonce as used                                |
  |  6. Check: humanNullifier not revoked                 |
  |  7. Check: nonce in humanPubSignals == sessionNonce    |
  |  8. Check: nonce in agentPubSignals == sessionNonce    |
  |  8. Check: humanMerkleRoot == humanTree.root()        |
  |  9. Check: agentMerkleRoot in rootHistory[30]         |
  | 10. Verify Groth16 proof via IGroth16Verifier         |
  | 11. Verify PLONK proof via IPlonkVerifier             |
  | 12. Emit HandshakeVerified(humanNull, agentNull, nonce)  |
  +-------------------------------------------------------+
```

Step 4.1 — Session Nonce Agreement: The human and agent agree on a fresh random session nonce. The session nonce may be generated by any of: (a) the verifier or relayer selecting a cryptographically random 256-bit value and distributing it to both parties; (b) a commit-reveal protocol where both parties contribute entropy (each commits to a partial nonce, reveals, and the final nonce is the hash of both contributions); (c) derivation from a blockchain randomness beacon (e.g., a VRF output from a recent block) combined with a per-session salt; or (d) a recent block hash combined with a commit-reveal hardening step to prevent miner manipulation. In all cases, the nonce is a public input to both circuits, binding both proofs to the same session. Freshness is enforced on-chain by the `usedNonces` mapping: a nonce that has been consumed by a prior `verifyHandshake` call is permanently rejected.

Step 4.2 — Human Proof Generation: The human generates a Groth16 proof using the HumanUniqueness circuit. The proof demonstrates membership in the humanTree, computes a scope-bound nullifier for Sybil detection, and binds the proof to the session nonce.

Step 4.3 — Agent Proof Generation: Concurrently, the agent generates a PLONK proof using the AgentPolicy circuit. The proof demonstrates that the agent holds a valid, non-expired credential in the agentTree with permissions satisfying the required scope mask, and binds the proof to the same session nonce.

Step 4.4 — On-Chain Batch Verification: A single transaction to the IdentityRegistry contract's `verifyHandshake()` function receives both proofs and the session nonce. The contract performs the following ordered checks:

1. **Nonce freshness:** The session nonce must not appear in the `usedNonces` mapping. If fresh, it is immediately marked as used.
2. **Nonce equality enforcement:** The contract verifies that the session nonce embedded in each proof's public signals matches the transaction argument: `humanPubSignals[4] == sessionNonce` and `agentPubSignals[5] == sessionNonce`. This prevents an attacker from submitting proofs generated for different sessions.
3. **Revocation status:** The human's nullifier hash is checked against the `humanRevocations` mapping. Agent revocation is enforced at the Merkle tree level: a revoked agent's credential commitment is set to zero in the agentTree (via LeanIMT update), causing subsequent Merkle proofs against that credential to fail verification.
4. **Human root validity:** The human proof's output Merkle root must match the current `humanTree` root.
5. **Agent root validity:** The agent proof's output Merkle root must appear in the 30-entry `agentRootHistory` circular buffer, accommodating tree updates during proof generation.
6. **Groth16 verification:** The human proof is verified by calling the deployed Groth16 verifier contract (generated by `snarkjs zkey export soliditycerifier`). The proof is decomposed into elliptic curve points (pA, pB, pC) in the BN128 pairing format and verified against the five public signals (humanMerkleRoot, nullifierHash, nonceBinding, scope, sessionNonce).
7. **PLONK verification:** The agent proof is verified by calling the deployed PLONK verifier contract. The 24-element proof array and six public signals (agentMerkleRoot, nullifierHash, scopeCommitment, requiredScopeMask, currentTimestamp, sessionNonce) are verified.

If all checks pass, the contract emits a `HandshakeVerified` event indexed by both nullifier hashes and the session nonce. Total gas cost is approximately 570,000 gas on the target Layer 2 chain.

Mixed Proving System Rationale: Groth16 is used for the human circuit because it enables reuse of the Semaphore v4 Phase 1 Powers of Tau ceremony, avoiding the operational burden of a new trusted setup for a solo-founder project. Only a circuit-specific Phase 2 ceremony is required. PLONK is used for the agent and delegation circuits because these circuits are expected to evolve more frequently (new credential fields, new permission schemes), and PLONK's universal setup avoids a new ceremony for each circuit update. The on-chain verifier accommodates both proof types through separate verifier contract interfaces (`IGroth16Verifier` and `IPlonkVerifier`) called within the same `verifyHandshake()` transaction.

5. Privacy-Preserving Composable Delegation Chain

The delegation mechanism 500 enables a human to delegate scoped permissions to a first AI agent (Agent A), which may further delegate narrower permissions to a second AI agent (Agent B), and so on up to a maximum chain depth of 3. At each hop, the delegation is proven in zero knowledge using the Delegation circuit, and the actual permission bitmask at each hop remains private — only a Poseidon hash commitment of the bitmask is published as a public output.

`FIGURE 5: Delegation Chain with Scope Commitment Linking (500)`

```
  Human/Agent A                  Agent A -> Agent B            Agent B ->
Agent C
  (AgentPolicy or                (Delegation Hop 0)           (Delegation Hop
1)
   prior hop)

  scopeCommitment_0              scopeCommitment_1
scopeCommitment_2
  = Poseidon2(scope_A, credCmt_A)          = Poseidon2(scope_B, credCmt_B)
= Poseidon2(scope_C, credCmt_C)
  (PUBLIC OUTPUT)                (PUBLIC OUTPUT)              (PUBLIC OUTPUT)
         |                              |                            |
         |  CHAIN LINK                  |  CHAIN LINK                |
         v                              v                            v
  +-------------------+         +-------------------+         +---------------
----+
  | Delegation Circuit|         | Delegation Circuit|         | (next hop or
|
  |                   |         |                   |         |  final
verifier)  |
  | PUBLIC INPUT:     |         | PUBLIC INPUT:     |         +---------------
----+
  |  previousScope    |         |  previousScope    |
  |  Commitment =     |         |  Commitment =     |
  |  scopeCmt_0       |         |  scopeCmt_1       |
  |                   |         |                   |
  | PRIVATE INPUTS:   |         | PRIVATE INPUTS:   |
  |  delegatorScope   |         |  delegatorScope   |
  |  = scope_A        |         |  = scope_B        |
  |  delegateeScope   |         |  delegateeScope   |
  |  = scope_B        |         |  = scope_C        |
  |                   |         |                   |
  | PROVES:           |         | PROVES:           |
  | 1. hash(scope_A,  |         | 1. hash(scope_B,  |
  |    credCmt_A)     |         |    credCmt_B)     |
  |    == scopeCmt_0   |         |    == scopeCmt_1   |
  | 2. scope_B AND    |         | 2. scope_C AND    |
  |    NOT(scope_A)   |         |    NOT(scope_B)   |
  |    == 0 (subset)  |         |    == 0 (subset)  |
  | 3. expiry_B <=    |         | 3. expiry_C <=    |
  |    expiry_A       |         |    expiry_B       |
  | 4. cumulative bit |         | 4. cumulative bit |
  |    invariant holds|         |    invariant holds|
  |                   |         |                   |
  | PUBLIC OUTPUT:    |         | PUBLIC OUTPUT:    |
  |  newScopeCmt =    |         |  newScopeCmt =    |
  |  scopeCmt_1       |         |  scopeCmt_2       |
  +-------------------+         +-------------------+
```

Step 5.1 — Identity-Bound Scope Commitment Chain Linking: The delegation chain is linked through identity-bound Poseidon hash commitments. The AgentPolicy circuit (Section 3) outputs `scopeCommitment = Poseidon2(permissionBitmask, credentialCommitment)` as a public signal. By including the credential commitment in the hash, the scope commitment is bound to a specific agent's identity, not just to a set of permission bits. This prevents impersonation: an actor with the same permission bits but a different credential cannot produce a matching scope commitment.

The first delegation hop takes this `scopeCommitment` as its `previousScopeCommitment` public input. Inside the Delegation circuit, the delegator's actual scope and credential commitment (both private inputs) are hashed using Poseidon and constrained to equal the `previousScopeCommitment`:

```
Poseidon2(delegatorScope, delegatorCredCommitment) ===
previousScopeCommitment;
```

This constraint is the identity-bound chain-linking mechanism. It proves that the delegator's private scope bits AND identity are consistent with what the previous hop publicly committed to, without revealing either.

The circuit then computes a new scope commitment for the delegatee: `newScopeCommitment = Poseidon2(delegateeScope, delegateeCredCommitment)`, which becomes the public input for the next hop. Each commitment in the chain binds both scope and identity.

Step 5.2 — Privacy-Preserving Subset Enforcement: Both the delegator's scope and the delegatee's scope enter the circuit as private inputs. The circuit decomposes both into individual bits using `Num2Bits(64)` and enforces, for each bit position i:

```
delegateeScopeBits.out[i] * (1 - delegatorScopeBits.out[i]) === 0;
```

This constraint ensures that every permission bit set in the delegatee's scope must also be set in the delegator's scope — the delegatee cannot gain permissions the delegator does not hold. The actual bit values are never exposed as public signals; only the hash commitments are published.

Step 5.3 — Expiry Narrowing: A `LessEqThan(64)` comparator constrains `delegateeExpiry <= delegatorExpiry`, preventing a delegation from extending the time validity beyond what the delegator possesses.

Step 5.4 — Delegation Authorization: The delegator signs a delegation token computed as a four-input Poseidon hash of the previous scope commitment, the delegatee's credential commitment, the delegatee's permission bitmask, and the delegatee's expiry timestamp, using EdDSA over the Baby Jubjub curve. The circuit verifies this signature, proving that the delegator explicitly authorized this specific delegation to this specific delegatee with these specific narrowed permissions and expiry.

Step 5.5 — Delegation Nullifier: A delegation nullifier is computed as `Poseidon2(delegationTokenHash, sessionNonce)`, providing per-session replay protection for delegation proofs.

The compiled Delegation circuit has 10,769 R1CS constraints and uses PLONK.

6. Cumulative Bit Encoding for Hierarchical Permissions

The permission bitmask uses a cumulative encoding scheme for hierarchical permission tiers. In the preferred embodiment, the financial transaction tier hierarchy is:

Bit	Permission	Implies
Bit 2	Financial transaction (< $100)	—
Bit 3	Financial transaction (< $10,000)	Bit 2
Bit 4	Financial transaction (unlimited)	Bits 2 and 3

The Delegation circuit enforces these invariants on the delegatee's scope using three circuit constraints:

```
// bit 4 set implies bit 3 must be set
delegateeScopeBits.out[4] * (1 - delegateeScopeBits.out[3]) === 0;

// bit 4 set implies bit 2 must be set
delegateeScopeBits.out[4] * (1 - delegateeScopeBits.out[2]) === 0;

// bit 3 set implies bit 2 must be set
delegateeScopeBits.out[3] * (1 - delegateeScopeBits.out[2]) === 0;
```

This encoding ensures that when delegation narrows permissions via bitwise AND, the resulting permission set remains logically consistent. For example, if a delegator has bit 4 (unlimited financial) and delegates only bit 3 ($10,000 limit) to a delegatee, the cumulative invariant ensures bit 2 ($100 limit) is also present, so the delegatee can execute transactions at all tiers up to the delegated limit.

Without cumulative encoding, bitwise AND delegation narrowing could produce inconsistent states — e.g., a delegatee might receive bit 3 ($10,000 financial) without bit 2 ($100 financial), creating an impossible permission set where the agent can approve $10,000 transactions but not $100 ones.

7. On-Chain Registry Contract

The IdentityRegistry contract 130 is implemented in Solidity 0.8.24 and deployed on a Layer 2 blockchain. Key implementation details:

7.1 — Dual Merkle Trees: The contract maintains two separate `LeanIMTData` structures (from `@zk-kit/lean-imt.sol`): `humanTree` for human identity commitments and `agentTree` for agent credential commitments. Both support a maximum depth of 20 (approximately 1,048,576 leaves). The use of LeanIMT (Lean Incremental Merkle Tree) provides gas-efficient incremental insertion without requiring pre-computation of the full tree.

7.2 — Root History Buffer: The contract maintains a circular buffer of the last 30 `agentTree` Merkle roots. When a new agent is enrolled, the resulting root is recorded in the buffer, and the mapping `agentRootExists` is updated. When the buffer wraps (after 30 insertions), the oldest root is removed from the mapping. This accommodates proof staleness: an agent that generates a proof against a Merkle root that is subsequently updated (e.g., by a new enrollment) can still verify its proof as long as the root is within the last 30 updates.

7.3 — Verifier Contract Interfaces: The IdentityRegistry interacts with three separately deployed verifier contracts: - `IGroth16Verifier`: Accepts Groth16 proof components (pA[2], pB[2][2], pC[2]) and 5 public signals for HumanUniqueness. - `IPlonkVerifier`: Accepts a 24-

element PLONK proof array and 6 public signals for AgentPolicy. - `IDelegationVerifier`: Accepts a 24-element PLONK proof array and 4 public signals for Delegation.

7.4 — Revocation: The contract maintains a revocation mapping for human identities (`humanRevocations`, keyed by nullifier hash). Human revocation is checked during handshake verification. Agent revocation is handled at the Merkle tree level: the contract owner updates the agent's credential commitment to zero in the agentTree via LeanIMT's update function, which changes the tree root and invalidates Merkle proofs against the post-revocation root. Proofs against pre-revocation roots that remain within the 30-entry agentRootHistory buffer continue to verify during a grace window until those roots are evicted by subsequent tree updates; this accommodates in-flight proofs that were generated before revocation reached the chain. This approach avoids the problem of using session-specific nullifiers as revocation keys.

7.5 — Chain-State Record: The contract maintains a chain-state record `lastScopeCommitment` implemented as a mapping from session nonce to a single field-element commitment. This record is the on-chain anchor for composable delegation chains and is written under two and only two conditions: (a) upon successful verification of a mutual handshake in `verifyHandshake`, the contract writes the agent's identity-bound scope commitment (output by the AgentPolicy circuit as a public signal) into the record keyed by the session nonce, thereby seeding the chain; and (b) upon successful verification of a delegation hop in `verifyDelegation`, the contract advances the record by writing the delegation's new scope commitment (output by the Delegation circuit as a public signal) into the same key, thereby extending the chain by one hop. The contract exposes no function that accepts a caller-supplied value for writing into `lastScopeCommitment`, and no code path other than the two enumerated above modifies the record. This design prevents confused-deputy attacks in which a malicious relayer would otherwise supply an arbitrary previous-scope-commitment value to link unrelated delegation proofs into a single chain, and prevents cross-session chain forking in which a delegation proof from one session would otherwise be accepted as continuing another session's chain.

7.6 — Batch Enrollment: The contract supports batch enrollment of both human identities (`enrollHumanBatch`) and agent credentials (`enrollAgentBatch`) for gas-efficient initialization.

8. Delegation On-Chain Verification

Delegation proofs are verified through the `verifyDelegation()` function, which processes a single delegation hop per call. The function accepts the proof, public signals, and session nonce as arguments. It does NOT accept a caller-supplied expected-previous-scope-commitment parameter; chain continuity is instead derived exclusively from on-chain state. For each call, the contract performs the following checks in order:

1. **Handshake prerequisite:** The function reverts with `DelegationRequiresHandshake` if the session nonce has not been previously consumed by a successful `verifyHandshake` call, as indicated by the on-chain `usedNonces` mapping. This binds every delegation chain to an authenticated mutual handshake.
2. **On-chain chain-linking verification:** The function reads `lastScopeCommitment[sessionNonce]` from the on-chain chain-state record (Section 7.5) and compares it to the proof's `previousScopeCommitment` public signal. A

mismatch reverts with `ScopeChainMismatch`. The expected previous-scope-commitment value is never accepted from the caller; it is read from on-chain state seeded by the handshake and advanced by each prior successful delegation.

3. **Nonce equality enforcement:** The proof's session nonce public signal must equal the function argument, binding the delegation to the same session as the handshake.
4. **Delegation nullifier replay protection:** The proof's delegation nullifier is checked against `usedDelegationNullifiers` and stored if fresh, preventing delegation replay.
5. **Hop count enforcement:** A `delegationHopCount` mapping tracks hops per session nonce. The function increments and checks against `MAX_DELEGATION_HOPS` (3), reverting with `MaxDelegationHopsExceeded` if the maximum chain depth is exceeded.
6. **Proof verification:** The delegation proof is verified via the deployed DelegationPlonkVerifier contract.
7. **Chain-state advancement:** Upon successful verification, the function writes the proof's `newScopeCommitment` public signal into `lastScopeCommitment[sessionNonce]`, making it the authoritative previous-scope-commitment for any subsequent delegation hop on the same session.

Delegation hops within a given session nonce must be submitted in strict sequential order. Concurrent or out-of-order submission will result in rejection due to chain-state mismatch.

9. Cryptographic Primitives

9.1 — Poseidon Hash Function: All hash computations within the circuits use the Poseidon algebraic hash function, which is optimized for arithmetic circuits over prime fields. Poseidon operates over the BN128 scalar field (p = 21888242871839275222246405745257275088548364400416034343698204186575808495617) and uses S-box exponentiation with x^5, partial-round optimization with MDS (Maximum Distance Separable) matrices, and configurable width (1 to 4 inputs in this system). Poseidon is approximately 8x more constraint-efficient than SHA-256 or Keccak-256 in arithmetic circuits.

9.2 — EdDSA on Baby Jubjub: Identity keys and operator signatures use the EdDSA signature scheme on the Baby Jubjub twisted Edwards curve (a = 168700, d = 168696) embedded in the BN128 scalar field. The Poseidon-based EdDSA variant (`EdDSAPoseidonVerifier` from circomlib) hashes the message with Poseidon rather than SHA-512, maintaining circuit-friendliness throughout.

9.3 — BN128 Elliptic Curve Pairing: Groth16 proof verification uses the BN128 (alt_bn128) pairing-friendly elliptic curve, which is natively supported by Ethereum precompiled contracts (EIP-196, EIP-197), enabling gas-efficient on-chain verification.

9.4 — Lean Incremental Merkle Tree: Both identity trees use the LeanIMT structure, which stores only the rightmost path from each leaf to the root, achieving O(log n) storage and O(log n) insertion cost. The tree depth of 20 supports 2^20 = 1,048,576 leaves.

CLAIMS

What is claimed is:

Claim 1. A computer-implemented method for privacy-preserving mutual authentication between a human user and an artificial intelligence agent, comprising: (a) maintaining, on a blockchain, a first Lean Incremental Merkle Tree storing human identity commitments computed as Poseidon hashes of EdDSA public key coordinates on the Baby Jubjub elliptic curve, and a second Lean Incremental Merkle Tree storing AI agent credential commitments computed as Poseidon hashes of agent credential fields including an agent's permission bitmask; (b) receiving, from the human user, a first zero-knowledge proof generated using a first zero-knowledge proving system, the first zero-knowledge proof proving Merkle membership of the human user's identity commitment in the first Merkle tree, computing a nullifier as a Poseidon hash of a scope identifier and a secret scalar, and computing a nonce binding as a Poseidon hash of the nullifier and a session nonce, without revealing the human user's identity; (c) receiving, from the AI agent, a second zero-knowledge proof generated using a second zero-knowledge proving system, the second zero-knowledge proof proving Merkle membership of the AI agent's credential commitment in the second Merkle tree, verifying an EdDSA signature of an operator over the credential commitment, performing Num2Bits(64) range checks on the permission bitmask and an expiry timestamp to prevent field overflow, enforcing bit-by-bit that the permission bitmask satisfies a required scope policy, verifying the credential has not expired via a LessThan(64) comparator, and outputting, as a public signal, an identity-bound scope commitment computed as a Poseidon hash of the permission bitmask and the AI agent's credential commitment that is Merkle-included in the second Merkle tree, without revealing the AI agent's credential fields; (d) verifying both the first zero-knowledge proof and the second zero-knowledge proof in a single blockchain transaction, wherein the on-chain verifier: (i) enforces that the session nonce embedded in each proof's public signals equals the transaction's session nonce argument, (ii) checks the session nonce for freshness against a used-nonce mapping, and (iii) writes the identity-bound scope commitment from the second zero-knowledge proof into a storage variable of the verifier contract indexed by the session nonce, the storage variable being subsequently read by a delegation verification function to enforce chain continuity; and (e) emitting a verification event only if both proofs are valid, the session nonce is fresh and matches both proofs, and the human user's nullifier has not been revoked.

Claim 2. The method of claim 1, wherein the first proving system is Groth16 and the second proving system is PLONK.

Claim 3. The method of claim 2, wherein the first zero-knowledge proof reuses a Powers of Tau Phase 1 trusted setup ceremony from the Semaphore v4 protocol, and the second zero-knowledge proof uses a universal setup requiring no circuit-specific ceremony.

Claim 4. The method of claim 1, wherein the AI agent credential commitment is computed as a five-input Poseidon hash of a model identifier hash, both coordinates of the operator's EdDSA public key on the Baby Jubjub elliptic curve, a permission bitmask, and an expiry timestamp.

Claim 5. The method of claim 1, wherein the identity-bound scope commitment output by the second zero-knowledge proof is computed as a two-input Poseidon hash of the AI agent's permission bitmask and the AI agent's credential commitment, binding permission scope to agent identity to prevent impersonation in delegation chains.

Claim 6. The method of claim 1, wherein verifying the first zero-knowledge proof requires exact equality between the human user's Merkle root and the current root of the first Merkle tree, and verifying the second zero-knowledge proof requires membership of the AI agent's Merkle root in a root history maintained by the identity registry contract, such that the verifier contract enforces an asymmetric root freshness requirement across the two Merkle trees within the single blockchain transaction of step (d).

Claim 7. The method of claim 6, wherein agent credential revocation is performed by updating the revoked agent's credential commitment to a designated invalid value in the second Merkle tree via an incremental tree update operation that changes the tree root, such that proofs generated against pre-revocation roots verify only while those roots remain in the root history, and proofs generated against post-revocation roots fail Merkle membership verification due to the invalidated leaf value.

Claim 8. The method of claim 1, wherein the first zero-knowledge proof further outputs a nonce binding value computed as a Poseidon hash of a nullifier hash and the session nonce, binding the proof to both the human user's identity and the specific session.

Claim 9. A computer-implemented method for privacy-preserving delegation of scoped permissions through a chain of artificial intelligence agents using zero-knowledge proofs, comprising: (a) maintaining, on a blockchain, a Lean Incremental Merkle Tree storing agent credential commitments and a session-indexed on-chain chain-state record, the chain-state record comprising a mapping from session nonce to a field-element commitment, wherein the chain-state record is written under exactly two conditions: by a handshake verification function upon successful mutual handshake verification (to seed the chain with an identity-bound scope commitment output by the handshake's agent proof), and by the delegation verification function upon successful delegation proof verification (to advance the chain by one hop); (b) receiving a delegation request identifying a session nonce, and requiring that the session nonce was previously consumed by a verified mutual authentication handshake that established the initial scope commitment in the chain-state mapping; (c) receiving, as private inputs to a delegation zero-knowledge proof circuit, a delegator's actual permission bitmask, the delegator's credential commitment which is Merkle-included in said agent credential tree, a delegatee's permission bitmask, and a delegatee's credential commitment; (d) constraining, within the circuit, that a two-input Poseidon hash of the delegator's actual permission bitmask and the delegator's credential commitment equals a previous scope commitment public input that is derived on-chain from the chain-state mapping indexed by the session nonce, thereby linking the current delegation to both the preceding chain state and the specific Merkle-included delegator identity without revealing the delegator's actual permissions; (e) performing Num2Bits(64) range checks on both permission bitmasks and constraining, within the circuit, that the delegatee's permission bitmask is a bitwise subset of the delegator's permission bitmask by enforcing, for each bit position i, that delegateeBits[i] multiplied by (1 minus delegatorBits[i]) equals zero; (f) constraining, within the circuit, that the delegatee's expiry timestamp is less than or equal to the delegator's expiry timestamp using a LessEqThan(64) comparator; (g) verifying, within the circuit, a digital signature, verified within the delegation zero-knowledge proof circuit using a circuit-friendly signature verification algorithm on an elliptic curve embedded in the scalar field of the proof system's base curve, of the delegator over a delegation token computed as a Poseidon hash of the previous scope commitment, the delegatee's credential commitment, the delegatee's permission bitmask, and the delegatee's

expiry timestamp; (h) outputting, as a public signal, a new scope commitment computed as a two-input Poseidon hash of the delegatee's permission bitmask and the delegatee's credential commitment; and (i) verifying the delegation zero-knowledge proof on a blockchain by: (1) reverting if no handshake consumed the session nonce, (2) comparing the proof's previous scope commitment public signal to the on-chain chain-state mapping value and reverting on mismatch, (3) storing a delegation nullifier derived from a delegation token hash and the session nonce to prevent replay, (4) advancing the chain-state mapping to the new scope commitment such that a subsequent delegation hop must extend from this newly-written on-chain value, and (5) enforcing a maximum delegation hop count stored as an on-chain constant.

Claim 10. The method of claim 9, wherein the delegation nullifier is computed as a Poseidon hash of the delegation token hash and the session nonce, providing per-session replay protection that is independent of the handshake nullifiers.

Claim 11. The method of claim 9, wherein the maximum delegation hop count is an immutable on-chain constant set at contract deployment, and the blockchain verification function maintains a per-session counter incremented with each verified delegation hop, reverting if the counter exceeds said constant.

Claim 12. The method of claim 9, further comprising: enforcing a cumulative bit encoding invariant on the delegatee's permission bitmask, wherein a higher-tier permission bit implies the presence of all lower-tier permission bits in a defined hierarchy, the invariant being enforced by circuit constraints of the form: higherBit * (1 - lowerBit) === 0.

Claim 13. The method of claim 12, wherein the cumulative bit encoding invariant encodes a tiered permission hierarchy in which: a third permission bit representing authority at a third tier implies a second permission bit and a first permission bit; the second permission bit representing authority at a second tier implies the first permission bit; and the first permission bit represents authority at a first tier, wherein the tiers represent increasing levels of operational scope.

Claim 14. The method of claim 9, wherein each delegation hop in the chain is verified by a separate call to a delegation verification function on the blockchain, with chain linking enforced by the on-chain chain-state record matching each hop's previous scope commitment public input to the preceding hop's new scope commitment public output stored in said record.

Claim 14A. The method of claim 9, wherein each scope commitment in the delegation chain is an identity-bound scope commitment whose pre-image comprises both a permission bitmask and a credential commitment that is a leaf in the Lean Incremental Merkle Tree storing agent credential commitments, such that the chain-linking constraint of step (d) requires each delegator to prove knowledge of a credential commitment that is simultaneously (i) Merkle-included in the on-chain agent credential tree and (ii) a component of the Poseidon hash that produced the previous scope commitment written to the chain-state record, and wherein the delegation circuit further proves Merkle membership of the delegatee's credential commitment in said agent credential tree and outputs a computed delegatee Merkle root as a public signal verified against a root history buffer maintained by the identity registry contract, thereby requiring bilateral Merkle-included identity binding at each delegation hop.

Claim 15. A computer-implemented method for integrated privacy-preserving authentication and delegation across a heterogeneous population of human users and artificial intelligence agents, comprising: (a) performing a mutual authentication handshake between a human user and an AI agent by verifying, in a single blockchain transaction, a first zero-knowledge proof of the human user's Merkle membership in a human identity tree and a second zero-knowledge proof of the AI agent's Merkle membership in an agent credential tree, wherein the on-chain verifier enforces that a session nonce embedded in each proof's public signals equals a transaction-argument session nonce, and wherein the second zero-knowledge proof outputs as a public signal an identity-bound scope commitment computed as a collision-resistant algebraic hash of the AI agent's permission bitmask and the AI agent's credential commitment, the algebraic hash being a hash function whose round function is composed of field-arithmetic operations native to the scalar field of the proof system's base curve; (b) recording, upon successful completion of the handshake, the identity-bound scope commitment from step (a) in an on-chain chain-state record indexed by the session nonce, the chain-state record being a mapping from session nonce to a field-element commitment, wherein the handshake verification function of step (a) sets the chain-state record value for a given session nonce to the identity-bound scope commitment output as a public signal by the agent's zero-knowledge proof upon successful handshake verification, and the delegation verification function of step (c) advances the chain-state record value for the same session nonce to the new identity-bound scope commitment output as a public signal by the delegation zero-knowledge proof upon successful delegation verification, wherein each value written to the chain-state record is derived from a verified zero-knowledge proof's public signal output rather than from a caller-supplied commitment parameter; (c) for each delegation hop in a chain comprising at most a predetermined maximum number of hops, verifying a delegation zero-knowledge proof that (i) proves knowledge of a delegator permission bitmask and a delegator credential commitment whose two-input Poseidon hash equals the on-chain chain-state value for the session nonce, wherein the two-input Poseidon hash binds both the delegator's permission bitmask and the delegator's credential commitment into a single identity-bound scope commitment such that an entity holding identical permissions but a different credential commitment that is Merkle-included in a different leaf of the agent credential tree cannot reproduce a matching hash value and therefore cannot satisfy the chain-linking constraint, (ii) proves that a delegatee permission bitmask is a bitwise subset of the delegator permission bitmask with Num2Bits(64) range checks, (iii) proves that a delegatee expiry timestamp does not exceed the delegator expiry timestamp, (iv) proves an EdDSA signature of the delegator over a delegation token, and (v) outputs a new identity-bound scope commitment computed as a two-input Poseidon hash of the delegatee's permission bitmask and the delegatee's credential commitment, the new identity-bound scope commitment being written to the chain-state mapping as the new authoritative value for subsequent hops such that each link in the chain cryptographically binds a specific Merkle-included agent identity to the delegated permission scope; and (d) processing each delegation zero-knowledge proof through a delegation verification function of the verifier contract, wherein said function's external interface accepts only the delegation proof, its public signals, and the session nonce as parameters, and wherein execution of said function performs, in order: (i) reading a used-nonce mapping slot keyed by the session nonce and reverting when said slot indicates no prior handshake; (ii) reading a chain-state mapping slot keyed by the session nonce and reverting when the stored value does not match the proof's previous scope commitment public signal; (iii) reading a delegation-nullifier mapping slot keyed by the proof's delegation nullifier and reverting when said slot is already set, then writing said slot as set; (iv) reading a per-session hop-counter mapping slot keyed by the session nonce, reverting when the

value incremented by one exceeds an on-chain maximum delegation hops constant, and writing the incremented value; (v) invoking the delegation verifier contract with the proof and public signals; and (vi) writing the proof's new scope commitment public signal to the chain-state mapping slot, advancing the chain state for any subsequent delegation hop.

Claim 15A. The method of claim 15, wherein: the first zero-knowledge proof uses the Groth16 proving system over the BN128 pairing-friendly elliptic curve; the second zero-knowledge proof uses the PLONK proving system over the BN128 curve; all hash commitments in steps (a) through (d) use the Poseidon algebraic hash function; all signatures use EdDSA on the Baby Jubjub twisted Edwards curve embedded in the BN128 scalar field; the human identity tree and agent credential tree are each a Lean Incremental Merkle Tree with a maximum depth of 20; and the cumulative bit encoding invariant of the permission bitmask is enforced by circuit constraints at both agent credential proof time and delegation proof time.

Claim 16. A system for unified human and artificial intelligence agent identity management with privacy-preserving mutual authentication and composable delegation, the system comprising: a blockchain-based identity registry contract maintaining: a first Merkle tree storing human identity commitments computed as Poseidon hashes of EdDSA public keys on the Baby Jubjub elliptic curve; a second Merkle tree storing AI agent credential commitments computed as five-input Poseidon hashes of agent credential fields including a model identifier hash, both coordinates of an operator's EdDSA public key, a permission bitmask, and an expiry timestamp; a root history circular buffer storing the last N roots of the second Merkle tree; a nonce mapping for replay protection; and revocation mappings for both human identities and agent credentials; a first verifier contract configured to verify zero-knowledge proofs for human identity; a second verifier contract configured to verify zero-knowledge proofs for AI agent credentials; a third verifier contract configured to verify zero-knowledge proofs for delegation; a handshake verification function that receives a Groth16 proof and a PLONK proof bound to a common session nonce, enforces that the session nonce embedded in each proof's public signals equals the transaction argument, verifies both proofs in a single transaction through the first and second verifier contracts respectively, checks nonce freshness and human revocation status, and upon successful verification of both proofs, atomically writes the identity-bound scope commitment output by the PLONK proof as a public signal into a chain-state mapping of the identity registry contract indexed by the session nonce, the chain-state mapping being writable only by the handshake verification function to seed an initial chain anchor and by the delegation verification function to advance the chain, such that said chain-state mapping value serves as the sole on-chain source of the previous scope commitment required by the delegation verification function for chain-linking verification, and emits a verification event upon success; and a delegation verification function that receives a delegation proof for a single hop, verifies chain-linking correctness against an expected previous scope commitment, enforces session nonce equality, stores a delegation nullifier for replay protection, increments and checks a per-session hop counter against a maximum, verifies the proof through the third verifier contract, and emits a delegation event with the new scope commitment.

Claim 17. The system of claim 16, further comprising: a human identity circuit compiled in Circom 2.1.6 with a Merkle tree depth of 20, the circuit configured to: derive an EdDSA public key from a private secret scalar using Baby Jubjub point multiplication; compute an identity commitment as a Poseidon hash of the derived public key; prove Merkle membership of the identity commitment in the first Merkle tree; compute a nullifier as a Poseidon hash of a scope

identifier and the secret scalar; and compute a nonce binding as a Poseidon hash of the nullifier and a session nonce.

Claim 18. The system of claim 16, further comprising: an agent credential circuit compiled in Circom 2.1.6 with a Merkle tree depth of 20, the circuit configured to: perform Num2Bits(64) range checks on the permission bitmask, expiry timestamp, and current timestamp to prevent field overflow; compute a credential commitment as a five-input Poseidon hash including both coordinates of the operator's EdDSA public key; verify an EdDSA signature of the operator over the credential commitment; prove Merkle membership of the credential commitment in the second Merkle tree; enforce bit-by-bit permission scope satisfaction against a required scope mask; and output an identity-bound scope commitment as a two-input Poseidon hash of the permission bitmask and the credential commitment for delegation chain entry.

Claim 19. The system of claim 16, further comprising: a delegation circuit compiled in Circom 2.1.6, the circuit configured to: receive as a public input a previous scope commitment and as private inputs a delegator scope, a delegator credential commitment, a delegatee scope, a delegatee credential commitment, and a delegatee Merkle proof comprising sibling hashes and a proof index; constrain that the two-input Poseidon hash of the delegator scope and delegator credential commitment equals the previous scope commitment, providing identity-bound chain linking; prove Merkle membership of the delegatee credential commitment in the second Merkle tree by computing a Merkle root from the delegatee credential commitment and the delegatee Merkle proof, and outputting said computed Merkle root as a public signal for on-chain verification against a root history buffer of the second Merkle tree; constrain bit-by-bit that the delegatee scope is a subset of the delegator scope; enforce a cumulative bit encoding invariant on the delegatee scope; verify an EdDSA signature of the delegator over a delegation token; and output a new identity-bound scope commitment as a two-input Poseidon hash of the delegatee scope and delegatee credential commitment.

Claim 20. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of claim 1.

Claim 21. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of claim 9.

Claim 22. A non-transitory computer-readable medium storing instructions that, when executed by a processor, cause the processor to perform the method of claim 15.

---

10. Comparison with Existing Systems

Feature	IdentityOS	Semaphore v4	World AgentKit	Indicio ProvenAI
Unified human + agent identity	Yes	No (human only)	Yes (but Orb-dependent)	No (agent only)
Mutual ZKP handshake	Yes	No	No	No

Feature	IdentityOS	Semaphore v4	World AgentKit	Indicio ProvenAI
Privacy-preserving delegation	Yes (scope commitment chains)	No	No	No (cleartext VCs)
Cumulative bit encoding	Yes	N/A	N/A	N/A
Mixed proving system (Groth16 + PLONK)	Yes	Groth16 only	Unknown	N/A (not ZKP)
Hardware-free	Yes	Yes	No (Orb)	Yes
Government-ID-free	Yes	Yes	Yes	Varies

11. Figures Summary

- **Figure 1:** System Architecture showing human subsystem, agent subsystem, and IdentityRegistry contract
- **Figure 2:** HumanUniqueness circuit data flow (private/public inputs, computation steps, public outputs)
- **Figure 3:** AgentPolicy circuit data flow
- **Figure 4:** Mutual Handshake protocol sequence
- **Figure 5:** Delegation chain with scope commitment linking across multiple hops

12. Alternative Embodiments

The following alternative embodiments are contemplated:

1. **Recursive SNARK delegation:** In an alternative embodiment, iterative on-chain delegation verification is replaced with recursive proof composition (e.g., Nova folding, SuperNova, or Halo2 accumulation schemes) such that a multi-hop delegation chain produces a single succinct proof verifiable in a single on-chain call regardless of chain depth. The scope commitment chain-linking mechanism described in Section 5 would be preserved inside the recursive circuit, with each folding step consuming the prior step's scope commitment as a public input. This reduces gas cost from O(hops) to O(1) at the cost of longer proof generation time.

2. **Platform-signed agent attestations:** In an alternative embodiment, the `operatorSignature` field in the AgentPolicy circuit is replaced with or augmented by a signature from the AI model provider (e.g., OpenAI, Anthropic, Google) that directly attests to the agent's model identity, capability level, and behavioral constraints. The circuit's EdDSA verifier would accept either an operator key or a platform key, identified by a public `signerType` flag (0=operator, 1=platform). This shifts the trust root from operator self-attestation to platform attestation without changing the circuit topology.

3. **Behavioral sybil resistance:** In an alternative embodiment, the passphrase-derived human identity stub described in the HumanUniqueness circuit is replaced with a behavioral or cognitive challenge-response enrollment mechanism. The enrollment circuit would commit a behavioral biometric (e.g., keystroke dynamics hash, cognitive challenge-response pattern) into the identity commitment alongside the EdDSA public key, producing `identityCommitment = Poseidon3(Ax, Ay, behavioralHash)`. The behavioral hash provides a sybil-resistant uniqueness signal without requiring specialized hardware (no Orb) or government identification.

4. **Cross-chain verification:** In an alternative embodiment, the IdentityRegistry contract's `verifyHandshake()` and `verifyDelegation()` functions are deployed on multiple L2 chains (e.g., Base, Arbitrum, Polygon zkEVM) with a cross-chain message bridge (e.g., LayerZero, Hyperlane, or native L2 bridge) relaying the chain-state record between chains, enabling a handshake verified on Chain A to seed a delegation chain verified on Chain B.

5. **Off-chain verification with periodic checkpoints:** In an alternative embodiment, high-frequency agent-to-agent interactions are verified off-chain using a local verifier that holds a recent snapshot of the Merkle roots and chain-state record. Periodically (e.g., every N interactions or every T seconds), an on-chain checkpoint transaction commits the accumulated proof-verification results to the IdentityRegistry, providing eventual on-chain auditability without per-interaction gas costs. The security model degrades from per-interaction on-chain finality to periodic checkpoint finality, acceptable for low-value interactions where latency matters more than settlement speed.

These alternative embodiments are contemplated within the scope of the invention.

13. Working Code Reference

The inventor has working implementations of all disclosed circuits and contracts: - `circuits/src/HumanUniqueness.circom` — 16,409 constraints, Groth16 - `circuits/src/AgentPolicy.circom` — 20,832 constraints, PLONK - `circuits/src/Delegation.circom` — 10,769 constraints, PLONK - `contracts/contracts/IdentityRegistry.sol` — LeanIMT trees, handshake + delegation verification with nonce equality enforcement, delegation nullifier replay protection, chain-linking verification, and hop count enforcement - 39 passing tests with real proof generation and on-chain verification at ~570k gas

---

END OF PROVISIONAL PATENT APPLICATION